

contract — setting Customer contracts

The contract module facilitates the management of phone number payment methods offered by MewbileTech to its customers. MewbileTech offers three types of contracts to pay for a phone number: namely through a Term contract, a Month-to-Month contract, and a Prepaid contract. All contracts operate on a monthly cycle, and customers are billed monthly according to the terms of their selected contracts unless otherwise specified.

The module implements classes to define and manage the various types of available phone plans customers may need. As a result, this module enables other employees to quickly assign contracts to specific customer phone numbers without needing to know the details of the contract type.

API Module Content

The contract module defines the 3 following subclasses along with their methods in this section:

class **TermContract**(Contract)

A term contract, offered by MewbileTech, can be instantiated to a customer's phone number on request. The contract establishes a mutually agreed-upon duration for which the customer must use the contract and its associated phone numbers from the initialization of the contract to an agreed upon end date. The duration of the agreement can span any amount of time and even after the end date, the contract will only be cancelled if the customer decided to cancel the contract. Under this contract, customers benefit from reduced monthly fees, lower call rates per minute, and free minutes of calling, compared to other types of contracts. However, to establish this contract, customers are required to submit a predetermined deposit in the event that the contract is cancelled before the agreed upon ending date, in which case, the customer will forfeit the deposit to the company. In case, the customer cancels the contract after the contract's duration, the deposit must be responsibly returned to the customer.

The following represent the initializer of the TermContract subclass along with the 3 methods the subclass modifies from the superclass:

__init__(start, end)

The initializer method for the TermContract subclass instantiates a term contract for a customer's phone number.

This method accepts a start parameter which is the month and year that the term contract is established on a phone number. Furthermore, the method also accepts an end parameter which represents a month and year greater than the start parameter as the agreed upon end date of the term contract for a customer's phone number. This method will return *None* as it initializes a term contract.

new_month(month, year, bill)

The method will cycle a customer's bill to the next corresponding month and year when called upon. As a result, this also means that this method will be used to initialize the first month and year of new customer term contracts where the term deposit will be paid by the customer.

The method accepts a month parameter that signifies which of the 12 months of the year that the term contract is in. Furthermore, the method also accepts a year parameter that tracks the year the term contract is in according to a calendar. Lastly, the method accepts a bill parameter which is the price of the customer's contract each month.

In order for the monthly bill to be applied, it is expected that the customer will have paid off the bill of the previous month when the method is called, in which case, the method will return *None* as its purpose is to track the date of the contract and modify the bill of a customer's term contract each new month.

bill_call(call)

This method will calculate the number of minutes a customer has used the phone number to call under a term contract. As a term contract offers a set number of free minutes used for calling, these minutes will first be subtracted before each minute will be subjected to the rate of calling set by the term contract.

A call parameter is accepted by the method which holds the records of outgoing and incoming calls and their durations made by the customer on the particular phone number.

The return value for this method is *None* as its purpose is to modify the monthly bill of the customer's contract with this phone number and add accordingly the amount the customer must pay.

cancel_contract()

When the customer wishes to cancel a term contract for a particular phone number, this method will terminate the term contract and return a float that represents the bill that the customer must pay in the month and year that the contract is cancelled. Similarly, the deposit will be included in this bill if applicable.

class **MTMContract**(Contract)

A Month-to-Month (MTM) contract, offered by MewbileTech, can be instantiated to a customer's phone number on request. An MTM contract starts when the customer agrees to attach their phone number to this contract and can be cancelled at any time, however, customers must pay higher monthly fees, higher cost per minute of calling, and have no free minutes. On the other hand, customers do not need to pay a deposit.

The following represent the initializer of the MTMContract subclass along with the 1 method the subclass modifies from the superclass:

__init__(start)

The initializer method for the TermContract subclass instantiates a term contract for a customer's phone number.

Moreover, the method accepts a start parameter which is the month and year that the MTM contract is established on a phone number and will return *None* as the method initializes a MTM contract.

new_month(month, year, bill)

The method will cycle a customer's bill to the next corresponding month and year when called upon. As a result, this also means that this method will be used to initialize the first month and year of new customer MTM contracts.

The method accepts a month parameter that signifies which of the 12 months of the year that the MTM contract is in. Furthermore, the method also accepts a year parameter that tracks the year the MTM contract is in according to a calendar. Lastly, the method accepts a bill parameter which is the price of the customer's contract each month.

In order for the monthly bill to be applied, it is expected that the customer will have paid off the bill of the previous month when the method is called, in which case, the method will return *None* as its purpose is to track the date of the contract and modify the bill of a customer's MTM contract each new month.

class **PrepaidContract**(Contract)

A prepaid contract, offered by MewbileTech, can be instantiated to a customer's phone number on request. A prepaid contract uses a credit system where the customer must deposit any amount of money into a balance to use for outgoing calls which is carried over each month. Consequently, a negative balance signifies that the customer has credit in the contract and a positive balance means the customer owes money. In either situation, the customer will be billed monthly according to how much they owe and \$25 dollars in credit will automatically be added to their balance in case the customer has under \$10 dollars in credit. Additionally, this contract has a very low cost per minute of calling without any free minutes and does not require the customer to fulfill any contractual agreements. In effect, the customer may cancel their prepaid contract at any time, however, if the credit is negative, the customer will not receive any remuneration, but if the credit is positive, the customer must pay the company the corresponding amount at cancellation.

The following represent the initializer of the PrepaidContract subclass along with the 2 methods the subclass modifies from the superclass:

__init__(start, balance)

The initializer method for the TermContract subclass instantiates a term contract for a customer's phone number.

Furthermore, the method accepts a start parameter that represents the month and year that a prepaid contract is established on a customer's phone number. In addition, the method also accepts a balance parameter which tracks the amount of credit a customer has with their contract to use for their outgoing calls. Since this method initializes a prepaid contract, it will return None.

new_month(month, year, bill)

This method will cycle a customer's bill to the next corresponding month and year when called upon. As a result, this also means that this method will be used to initialize the first month and year of new customer prepaid contracts.

The method accepts a month parameter that signifies which of the 12 months of the year that the prepaid contract is in. Furthermore, the method also accepts a year parameter that tracks the year the prepaid contract is in according to a calendar. Lastly, the method accepts a bill parameter which is the price of the customer's balance each month.

The customer does not have to pay a monthly bill, but \$25 dollars will be added to the balance as long as the customer has under \$10 dollars in credit. In any case, the method will return *None* as its purpose is to track the date of the contract and modify the bill of a customer's prepaid contract each new month.

cancel_contract()

When the customer wishes to cancel the prepaid contract of a phone number, this method will be called and will return a float representing the total that the customer must pay if the balance is greater than 0 at cancellation. Otherwise, if the customer has credit at cancellation, the remaining amount will not be refunded.

Implementation Module Content

Each of the 3 subclasses discussed above have methods with parameters that will be further discussed in this section. Those without parameters will be omitted:

class **TermContract**(Contract)

__init__(start, end)

Start: A parameter that tracks the starting month and year of a term contract. The parameter accepts type datetime arguments for only months and years from the datetime module and must be of a value older than the end parameter.

End: A parameter that tracks the ending month and year of a term contract. The parameter accepts type datetime arguments for only months and years from the datetime module and must be of a value more recent than the start parameter.

new_month(month, year, bill)

Month: A parameter that tracks the increments in months whenever a new month is cycled for a customer's bill. This parameter accepts integers, meaning that the month to be inputted must be converted to a number.

Year: A parameter that tracks the increments in years whenever the 12th month of the year is cycled for a customer's bill. This parameter must accept the entire calendarial year as an input otherwise, adverse effects to the code may occur.

Bill: This parameter records the bill that a customer will accrue monthly and is of type Bill object from the Bill module. This parameter defines the monthly fee of a term contract, its free minutes, its rate per calling minute, and the price of the fixed deposit, adding all applicable expenses to a customer's monthly bill.

In this method, it is expected that a bill has already been created for the current month and year and that the bill parameter is in the correct month and year. Similarly, customers are expected to have paid their previous month's expenses. Furthermore, parameters month and year will be compared against parameter end from the initializer every time this method is called to verify whether the contract deadline has passed.

bill_call(call)

Call: This parameter accepts type Call objects from the call module which is an object that has recorded all incoming and outgoing calls as well as their durations made by a customer's phone number. All outgoing call durations will be calculated and added to the bill parameter which will calculate the applicable expenses of the customer's calls.

class **MTMContract**(Contract)

__init__(start)

Start: A parameter that tracks the starting month and year of a term contract. The parameter accepts type datetime arguments for only months and years from the datetime module.

new_month(month, year, bill)

Month: A parameter that tracks the increments in months whenever a new month is cycled for a customer's bill. This parameter accepts integers, meaning that the month to be inputted must be converted to a number.

Year: A parameter that tracks the increments in years whenever the 12th month of the year is cycled for a customer's bill. This parameter must accept the entire calendarial year as an input otherwise, adverse effects to the code may occur.

Bill: This parameter records the bill that a customer will accrue monthly and is of type Bill object from the Bill module. This parameter defines the monthly fee of a MTM contract, and its rate per calling minute, adding all applicable expenses to a customer's monthly bill.

In this method, it is expected that a bill has already been created for the current month and year and that the bill parameter is in the correct month and year. Similarly, it is expected that customers will have paid off their previous month's expenses at the beginning of each new month.

class **PrepaidContract**(Contract)

__init__(start, balance)

Start: A parameter that tracks the starting month and year of a term contract. The parameter accepts type datetime arguments formatted for months and years from the datetime module.

Balance: A parameter that records the amount of money that a customer can use or has spent on outgoing calls. This parameter accepts integers as an input and it is expected that customers do not provide negative amounts of money.

new_month(month, year, bill)

Month: A parameter that tracks the increments in months whenever a new month is cycled for a customer's bill. This parameter accepts integers, meaning that the month to be inputted must be converted to a number.

Year: A parameter that tracks the increments in years whenever the 12th month of the year is cycled for a customer's bill. This parameter must accept the entire calendarial year as an input otherwise, adverse effects to the code may occur.

Bill: This parameter records the bill that a customer will accrue monthly and is of type Bill object from the Bill module. This parameter defines the rate per minute of calling, adding all applicable expenses to a customer's monthly bill, carrying the balance parameter to each new month.

In this method, it is expected that a bill has already been created for the current month and year and that the bill parameter is in the correct month and year.